

Part 1

1. GRPO vs PPO: GRPO removes the need for a learned value function (critic). Explain how GRPO estimates the baseline for advantage computation without a critic, and what tradeoffs this introduces.

The main difference between **PPO** and **GRPO** is how they compute the **baseline** used in the **advantage**. In standard **PPO**, the advantage is estimated using a learned **value function / critic**, typically through **GAE**, so the system must train both a **policy model** and a **value model**. DeepSeekMath points out that this is expensive for LLMs because the value model is usually comparable in size to the policy, which increases both memory and computation.

How GRPO estimates the baseline for advantage computation without a critic: for each prompt q , GRPO samples a **group** of outputs from the old policy, scores them with a reward function, and then computes the baseline from the **group statistics**. Concretely, it uses the **group mean reward** as the baseline and normalizes by the **group standard deviation**, so the advantage for output i is based on its **relative reward within the group** rather than on a learned value estimate. In the outcome-supervision version used in the paper, the normalized reward is then assigned to all tokens of that output. The illustrated GRPO post explains the same intuition very clearly: GRPO samples G outputs, computes rewards r_i , then sets $A_i = \frac{r_i - \bar{r}}{\sigma_r + \epsilon}$, where $\bar{r}$ and σ_r are the mean and standard deviation over the group.

The **tradeoff** is that GRPO is **more memory-efficient** and simpler than PPO, but it replaces a learned, token-aware baseline with a **relative group-based baseline**. This introduces several consequences. First, training now depends heavily on the **quality and diversity of sampled outputs** within each group: if all sampled answers are bad, the “best” one may still only be bad relative to the others. Second, GRPO requires generating **multiple completions per prompt**, which increases sampling cost and can reduce effective batch size. Third, because the baseline is **relative**, the signal is less about absolute quality and more about **ranking within the group**. The blog also highlights practical tradeoffs such as **generation cost**, **group size tuning**, and sensitivity to the **KL penalty** and **reward design**.

2. Reward Design: Discuss the difference between using a learned reward model vs. a rule-based reward function (e.g., checking if an answer matches ground truth). What are the risks of reward hacking in each case?

A **learned reward model** and a **rule-based reward function** differ mainly in **flexibility** versus **reliability**.

A **learned reward model** is useful when the target behavior is hard to specify with exact rules, such as **helpfulness, style, reasoning quality, or preference alignment**. It can evaluate outputs in a more nuanced way than a binary correctness check. In DeepSeekMath, the paper discusses reward models and also explores both **outcome supervision** and **process supervision**, where rewards can be given either at the final answer level or at intermediate reasoning steps.

A **rule-based reward function**, by contrast, is much simpler and often much cleaner when the task has an objective target. The illustrated GRPO post gives exactly this kind of example: for math, one can extract the final answer with a regex and compare it to ground truth; for coding, one can run tests in a sandbox; and one can also include a **format reward** for following a required structure. This is especially attractive for tasks like math because the reward is cheap, deterministic, and easier to audit.

The risk of **reward hacking** exists in both cases, but in different forms.

With a **learned reward model**, the classic failure mode is that the policy learns to exploit **imperfections in the reward model** rather than truly solving the task. For example, it may produce outputs that “look like” good reasoning or match stylistic patterns the reward model likes, even when the underlying reasoning is wrong. This is one reason PPO-style RLHF uses **KL regularization** to keep the policy from drifting too far from the reference model, and GRPO keeps a KL term as well.

With a **rule-based reward**, reward hacking is usually narrower but still real. The model may learn to optimize the **surface rule** rather than the intended behavior. For example, in math it might learn to game answer extraction formats, and in code it may overfit to the visible tests rather than solving the general problem. If a format score is included, the model may also learn to satisfy formatting requirements without improving correctness. The blog explicitly warns that GRPO performance depends heavily on **well-designed reward functions**, and poorly designed rewards can optimize the wrong objective.

So in practice, **rule-based rewards** are preferable when correctness is **verifiable**, while **learned reward models** are preferable when the objective is **complex or subjective**.

But neither is automatically safe: learned rewards are vulnerable to **model exploitation**, while rule-based rewards are vulnerable to **specification gaming**.

3. SFT vs. GRPO: In Homework 3, you fine-tuned a VLM using supervised fine-tuning (SFT) with LoRA. Compare the SFT approach with GRPO training: how does the training signal differ, and when would each be preferred?

Compared with the **SFT + LoRA** setup from Homework 3, **GRPO** uses a very different training signal.

In **SFT (Supervised Fine-Tuning)**, the model is trained by **imitating target outputs** directly. The signal is token-level cross-entropy on a reference answer: the model is told, in effect, “this is the correct sequence; increase its probability.” This is a **teacher-forcing** style objective, and it works well when we already have high-quality demonstrations. In Homework 3, this was a natural choice because the task was a supervised VLM classification/QA setting and the model could be adapted efficiently with **LoRA**.

In **GRPO**, the model is not trained to copy a target sequence token by token. Instead, it **samples multiple outputs**, receives a **scalar reward** for each one, converts those rewards into **relative advantages within a group**, and then updates the policy to make higher-reward outputs more likely. So the signal is **preference- or reward-based**, not direct imitation. DeepSeekMath shows this clearly: GRPO is applied after SFT, and it improves mathematical reasoning by optimizing outputs according to rewards rather than just supervised targets.

Because of this, the two methods are best suited to different situations.

I would prefer **SFT** when:

- I have **clean labeled data** or high-quality demonstrations.
- The target output is relatively well-defined.
- I want **stable, efficient training**.
- The task is narrow enough that imitation of good examples is sufficient.

I would prefer **GRPO** when:

- I care about **reasoning quality** or behaviors that are hard to teach by plain imitation.
- I have a **reward signal** but not necessarily one gold target sequence.
- I want to optimize for **final performance**, not just match the training distribution.
- I am fine with more expensive and less stable training in exchange for better task-specific improvement.

In practice, these are often **complementary rather than competing**. The DeepSeekMath pipeline itself follows this pattern: first do **SFT** to give the model a strong supervised starting point, then apply **GRPO** to push performance further using reward-based optimization.

Part 2: Implementing and Training GRPO (100 points)

Problem 1: GPU Verification and Library Installation

```

import torch

print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
print("CUDA device count:", torch.cuda.device_count())
if torch.cuda.is_available():
    print("GPU name:", torch.cuda.get_device_name(0))

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
t = torch.randn(2, 3, device=device)
KEY = 73
cipher_bytes = [14, 27, 25, 6, 105, 32, 58, 105, 8, 37, 37, 105, 16, 38, 60, 105, 7, 44, 44, 45]

if t.is_cuda:
    cipher = torch.tensor(cipher_bytes, dtype=torch.uint8, device=device)
    decoded = cipher ^ KEY
    secret_word = "".join(chr(c) for c in decoded.cpu().tolist())
    print(f"\nGPU check passed! Secret word: {secret_word}")
else:
    print("\nNo GPU detected. Please switch to an A100 runtime.")

```

```

...
===== 60.7/60.7 MB 44.8 MB/s eta 0:00:00
===== 697.4/697.4 kB 56.6 MB/s eta 0:00:00
===== 527.0/527.0 kB 47.5 MB/s eta 0:00:00
===== 47.6/47.6 MB 55.1 MB/s eta 0:00:00
===== 36.4/36.4 MB 71.0 MB/s eta 0:00:00

PyTorch version: 2.10.0+cu128
CUDA available: True
CUDA device count: 1
GPU name: NVIDIA A100-SXM4-40GB

GPU check passed! Secret word: GRPO is All You Need

```

Problem 3: Understanding GRPO (15 points)

1. In your own words, explain why the group-based advantage normalization works as a baseline. What happens when all G completions for a prompt receive the same reward?

Instead of estimating an absolute value for each output, GRPO compares outputs **within the same group**. The **mean reward** of the group acts as a **baseline**, and each output is evaluated based on its **relative advantage** compared to others. Outputs with higher-than-average reward receive positive advantage, while lower-than-average ones receive negative advantage.

This works because policy optimization mainly depends on **relative ranking**, not absolute reward values. As long as the model can distinguish which outputs are better than others for the same prompt, it can improve the policy without needing a separate critic. This makes GRPO more **memory-efficient** and simpler than PPO.

If all sampled completions receive the **same reward**, then there is no difference between them. In this case:

- All outputs have **zero advantage**
- The model receives **no gradient signal**
- The policy **does not update**

This is actually a reasonable outcome: if the reward function cannot distinguish between outputs, then there is **no useful learning signal**. However, it also highlights an important limitation: GRPO relies on **reward diversity within the group**. If rewards are too similar (due to weak reward design or low sample diversity), training can **stall**.

2. What is the role of the clipping term $\text{clip}(Q_{i,t}, 1-\epsilon, 1+\epsilon)$? What could go wrong during training without it?

The **clipping term** in GRPO (inherited from **PPO**) is used to ensure **stable policy updates** by limiting how much the new policy can deviate from the old one.

During training, the model updates its probability of generating certain tokens based on the advantage signal. However, without constraints, these updates can become too

large, causing the model to change its behavior too quickly. Clipping introduces a **trust region**, ensuring that updates remain **bounded and incremental**.

The role of clipping is therefore to:

- Prevent **overly large updates**
- Maintain **training stability**
- Avoid sudden **distribution shifts**

Without clipping, several problems can occur:

- **Unstable training / divergence**: The model may make excessively large updates, leading to exploding gradients or failure to converge.
- **Overfitting to high-reward samples**: A few high-reward outputs could dominate training, causing the model to overspecialize and lose generalization.
- **Policy collapse**: The model may collapse to narrow or repetitive behaviors if certain outputs are overly reinforced.
- **Loss of pretrained knowledge**: Large updates can push the model too far from its original distribution, degrading language quality and general capabilities.

Clipping effectively enforces the idea: *improve the policy toward better outputs, but **do not change too much in one step***.

This is especially important in GRPO, because it does not use a **critic** to stabilize training. As a result, clipping plays a critical role in ensuring that learning remains **controlled, stable, and reliable**.

Problem 7: Train with GRPO (20 points)

1. Report the hyperparameter settings you used to get the best model. How did the reward values change over training?

The best GRPO model was obtained with the following hyperparameters:

- num_generations = 8
- max_completion_length = 256
- learning_rate = 1e-5
- max_steps = 100
- epsilon = 0.2

- temperature = 0.6
- beta = 0.0

With this configuration, the final GRPO model achieved 75.0% accuracy and 100.0% format compliance on 8 held-out bean-leaf samples. The model consistently produced answers in the required Answer: format, and it also generated short step-by-step reasoning before the final label.

In terms of reward behavior, the format reward improved first and became stable relatively early, because the model quickly learned to follow the required response structure. By the end of training, this was reflected in the held-out evaluation, where format compliance reached 100%. The accuracy reward improved more gradually and remained noisier, since the harder part of the task was not producing the format, but correctly distinguishing between visually similar unhealthy classes, especially bean rust and angular leaf spot. In other words, the model learned the output structure faster than it learned the fine-grained disease classification boundary.

For this task, I found that **num_generations**, **max_completion_length**, and **temperature** were especially important. Increasing **num_generations** was helpful because GRPO relies on comparing multiple completions for the same prompt; with more completions per prompt, the relative reward signal became more informative, which reduced the tendency of the model to collapse to a single unhealthy label. **max_completion_length** also mattered because the model was required to produce both reasoning and a final answer after Answer:. If this value was too small, the output could be truncated before the final label, which directly harmed both the format reward and the accuracy reward. In addition, lowering **temperature** helped make the predictions more stable. With a high **temperature**, the model produced more random and inconsistent completions, while a lower temperature reduced unnecessary variability and made the final label more reliable, which was especially important for distinguishing between the two visually similar unhealthy classes.

The other hyperparameters were kept at their default values. For my beans classification task, I did not observe large performance changes from adjusting parameters such as epsilon, beta, or the LoRA dropout, so I kept those settings unchanged to avoid unnecessary complexity. Their roles are still important in general: epsilon controls clipping in the GRPO objective, beta controls KL regularization, and the LoRA parameters determine the capacity and regularization of the adapter layers. However, in my experiments, these had a smaller effect than num_generations, max_completion_length, temperature, and the prompt/output-format design.

2. Compare the training dynamics of GRPO (this homework) with SFT/LoRA (HW3). Which converged faster, and which produced better results on your dataset?

Compared with SFT/LoRA in HW3, GRPO showed slower and noisier training dynamics. This is expected, because SFT uses direct supervision with the correct answer at the token level, while GRPO relies on sparse reward signals computed from sampled completions. For a small 3-class visual classification task like bean disease recognition, SFT provides a stronger and denser learning signal, so it converges faster and more stably.

In my experiments, SFT/LoRA from HW3 appeared to produce better pure classification performance on this dataset, while GRPO was more helpful for enforcing the desired response style. In HW3, the fine-tuned SFT model performed very strongly on the evaluated held-out bean images and was better at directly predicting the correct class label. In contrast, GRPO in HW4 achieved 75.0% held-out accuracy, which is reasonable but still below the classification quality I observed from the SFT model. However, GRPO clearly improved output formatting behavior: the final GRPO model produced 100% format compliance, and every evaluated response followed the requested Answer: structure with explicit reasoning before the final label.

Overall, for this dataset, SFT/LoRA converged faster and gave better classification accuracy, while GRPO produced better alignment with the desired response format and reasoning style. This makes sense because the beans task is primarily a supervised fine-grained classification problem, where SFT is a natural fit, whereas GRPO is more useful for shaping how the model responds once the task itself is already reasonably learned.

Problem 8.3: Reflection (5 points)

Compare the GRPO-trained model with the pre-trained base model (zero-shot) and your SFT model from HW3:

- **Does the GRPO model use the Answer : format more consistently?**
- **Does it show step-by-step reasoning before the answer?**

Compared with the pre-trained base model (zero-shot) and the SFT model from HW3, the GRPO-trained model used the Answer: format much more consistently. In my final held-out evaluation, the GRPO model reached 100% format compliance, meaning every evaluated response included the required final Answer: line. This was a clear improvement over the zero-shot base model and also over the SFT model from HW3, since those models were primarily optimized for direct classification accuracy rather than for following a specific response format. The base model often produced unconstrained answers unless carefully prompted, and the SFT model was better at predicting the correct class label but did not naturally enforce the Answer: structure in the same consistent way.

The GRPO model also showed step-by-step reasoning before the final answer much more reliably than the other two models. In the held-out examples, every GRPO response included explicit reasoning followed by a final Answer: line, which matches the intended behavior of this homework. By contrast, the zero-shot base model and the HW3 SFT model were better viewed as classification models: they were more likely to output a short direct label, or required specially designed prompts to produce reasoning. Therefore, for this beans dataset, the main advantage of GRPO was not necessarily higher raw classification accuracy, but rather better alignment with the desired output style: structured reasoning plus a clearly formatted final answer. In comparison, the SFT model still appeared stronger for pure classification accuracy, while the zero-shot base model was the weakest overall.